

R E P O R T

제목



학 과	컴퓨터공학과
교수님	천세진 교수님
학 번	2020196
이 름	김지민
제출일	6/12



동아대학교
DONG-A UNIVERSITY

목차

1.데이터셋 설명

2.선택한 알고리즘 개요

3.구현 방식

4.실험 환경

5.파라미터 설정

6.정확도 비교 결과

6-1. Bloom Filter - False Positive Rate

6-2. Count-Min Sketch - 상대 오차

7.메모리 사용량 비교 결과

8.처리 시간 비교 결과

9.파라미터 비교 실험 결과

9-1. Bloom Filter - 목표 FP Rate에 따른 변화

9-2. Count-Min Sketch - width/depth에 따른 변화

10.알고리즘별 장단점 분석

11.최종 분석 질문

12.결론

소스 코드: <https://github.com/jimin-20/streaming-algorithms.git>

1. 데이터셋 설명

1.1 데이터셋

MovieLens 1M Dataset을 사용하였다.

MovieLens 1M은 6,040명의 사용자가 3,952개의 영화에 대해 남긴 1,000,209건의 평점 데이터를 포함한다.

본 실험에서는 ratings.dat 파일을 스트림 형태로 한 줄씩 읽어 처리하였다.

컬럼	설명
UserID	사용자 ID (1~6040)
MovieID	영화 ID (1~3952)
Rating	평점 (1~5)
Timestamp	Unix timestamp

1.2 스트림 정의

스트림 데이터는 다음 두 가지 관점에서 활용하였다.

(UserID, MovieID) 조합 → "이 사용자가 이 영화를 평가한 적이 있는가?" → Bloom Filter로 포함 여부를 근사 판정

MovieID의 등장 빈도 → "이 영화는 스트림에서 몇 번 평가되었는가?" → Count-Min Sketch로 빈도를 근사 추정

2. 선택한 알고리즘 개요

2-1. Bloom Filter

Bloom Filter는 원소의 집합 포함 여부를 근사적으로 판정하는 확률적 자료구조이다. 비트 배열(bit array)과 여러 개의 해시 함수를 사용하며, 원소를 추가할 때 각 해시 함수가 가리키는 비트를 1로 설정한다. 조회 시에는 해당 원소의 모든 해시 위치 비트가 1인지 확인하며, 하나라도 0이면 해당 원소는 집합에 "절대 존재하지 않음"을 의미한다. 반대로 모두 1이면 "존재할 가능성이 있음"을 의미하지만, 이는 다른 원소들의 해시 충돌로 인해 실제로는 존재하지 않는 데 존재한다고 잘못 판정되는 False Positive가 발생할 수 있다. 반면 False Negative는 절대 발생하지 않는다는 특징이 있다.

2-2. Count-Min Sketch

Count-Min Sketch는 스트림 내 각 항목의 등장 빈도를 근사적으로 추정하는 자료구조이다. $\text{depth} \times \text{width}$ 크기의 2차원 카운터 테이블을 사용하며, 원소를 추가할 때 depth개의 서로 다른 해시 함수를 통해 각 행(row)에서 하나의 열(column) 위치를 계산하여 해당 카운터를 1 증가시킨다. 빈도를 조회할 때는 depth개의 카운터 값 중 **최솟값(minimum)**을 추정값으로 사용하는데, 이는 해시 충돌로 인한 과대추정을 다른 행에서의 더 작은 값으로 보정하기 위함이다. 이 자료구조의 추정값은 항상 실제 빈도 이상이며, 절대 과소추정되지 않는 one-sided error 특성을 가진다.

3. 구현 방식

두 알고리즘은 외부 자료구조 라이브러리 없이 직접 구현 했으며, 해시 함수만 mh3 라이브러리를 사용하였다.

Bloom Filter 구현

비트 배열은 bitarray 라이브러리를 사용하였고, 비트 배열 크기(m)와 해시 함수 개수(k)는 예상 데이터 개수(n)와 목표 오답률(p)로부터 다음 공식을 이용해 계산하였다.

$$m = -(n \times \ln(p)) / (\ln(2))^2$$

$$k = (m / n) \times \ln(2)$$

해시 함수는 mmh3.hash()를 시드(seed)를 0과 1로 다르게 호출하여 두 개의 독립적인 해시 값(h1, h2)을 얻은 뒤, double hashing 기법((h1 + i × h2) mod m)으로 k개의 해시 위치를 생성하였다.

Count-Min Sketch 구현

width × depth 크기의 2차원 리스트를 카운터 테이블로 사용하였다. 각 행(depth)마다 서로 다른 시드의 mmh3.hash()로 열 위치를 계산하여 카운터를 증가시키며, 빈도 조회 시에는 depth개 카운터의 최솟값을 반환한다.

스트리밍 처리 및 Ground Truth

데이터를 한 줄씩 읽으면서 동시에 두 알고리즘에 반영함과 동시에, 비교를 위한 정확한 자료 구조(Ground Truth)도 함께 구축하였다.

알고리즘	Ground Truth	비교
Bloom filter	실제 set 기반(UserID, MovieID) 포함여부	False Positive Rate 측정
Count-Min Sketch	실제 dict 기반 MovieID별 정확한 빈도	상대 오차(relative error) 측정

Ground Truth는 비교를 위한 기준일 뿐, 실제 스트리밍 환경에서는 사용하지 않는(메모리를 모두 사용하는) 자료구조이다.

4. 실험 환경

언어/환경: Python 3, Google Colab (.ipynb)

주요 라이브러리: mmh3(해시 함수), bitarray(비트 배열), pympler(실제 메모리 측정), pandas, numpy, matplotlib

데이터 규모: 1,000,209건 (고유 (User, Movie) 조합 약 100만 개, 고유 MovieID 3,952개)

5. 파라미터 설정

Bloom Filter

기본 실험: 목표 False Positive Rate(p) = 0.01

파라미터 비교 실험: $p \in \{0.5, 0.1, 0.01, 0.001\}$

Count-Min Sketch

기본 실험: width = 2000, depth = 5

파라미터 비교 실험: (width, depth) $\in \{(500,3), (1000,4), (2000,5), (5000,5), (10000,7)\}$

6. 정확도 비교 결과

6-1. Bloom Filter - False Positive Rate

전체 1,000,209건의 (User, Movie) 조합을 Bloom Filter에 추가한 뒤, 스트림에 등장하지 않은 50,000개의 무작위 (User, Movie) 조합으로 질의를 수행하였다.

목표 FP Rate (설계값): 0.01

이론적 FP Rate (수식 기반): 0.0100

실측 FP Rate: 0.0096 (50,000개 질의 중 481개 오답)

Bloom Filter 크기: 9,587,061 bits, 해시 함수 개수: 7개

실행결과

목표 FP rate (설계값): 0.01

이론적 FP rate (수식 기반): 0.0100

실측 FP rate (50,000개 질의 중 481개 오답): 0.0096

이론값과 실측값이 거의 일치하여, 설계 단계에서 계산한 비트 배열 크기(m)와 해시 함수 개수(k)가 의도한 정확도를 정확히 달성함을 확인하였다.

6-2. Count-Min Sketch - 상대 오차

모든 고유 MovieID(3,952개)에 대해 Count-Min Sketch의 추정 빈도와 Ground Truth(정확한 빈도)를 비교하였다.

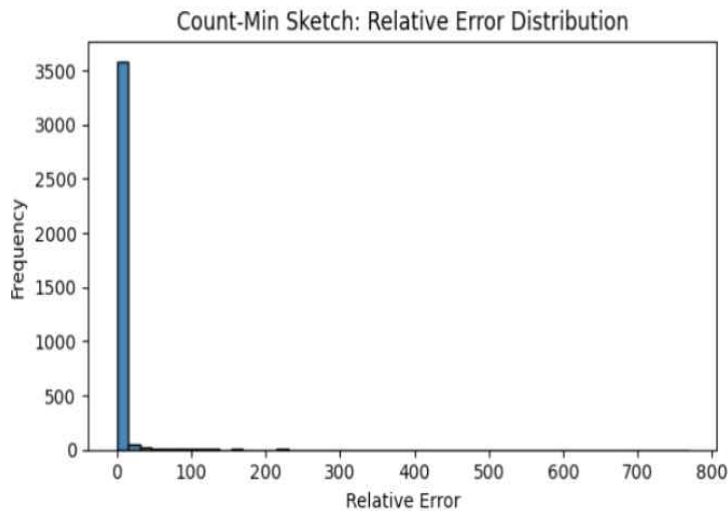
평균 상대 오차: 2.9958 (약 300%)

중앙값 상대 오차: 0.0000 (0%)

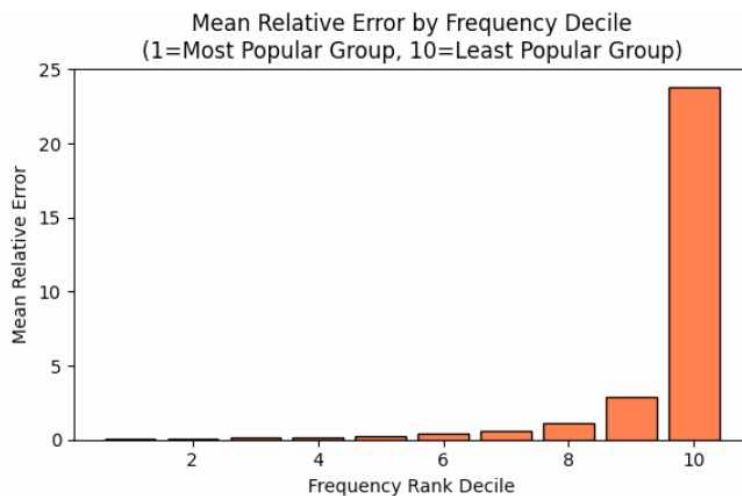
최대 상대 오차: 768.0000(76.800%)

평균 절대 오차: 39.64

Count-Min Sketch는 항상 실제값 이상으로 추정되므로(one-sided error) 모든 오차는 0 이상으로 나타났다. 빈도 구간별 분석 결과, 인기 영화(빈도 상위 그룹)일수록 상대 오차가 작고, 비인기 영화(빈도 하위 그룹)일수록 상대 오차가 크게 나타났다. 이는 Count-Min Sketch가 heavy hitter(상위 빈도 항목) 추정에 강점이 있는 자료구조임을 보여주는 내용이다.



Count-Min Sketch 추정 오차 분석. 전체 3,706개 MovieID에 대한 상대 오차 분포 히스토그램. 대다수(약 3,600개 이상)는 상대 오차가 0~10 범위에 집중되어 있으나, 일부 항목에서 최대 768까지의 극단적인 오차가 나타난다.



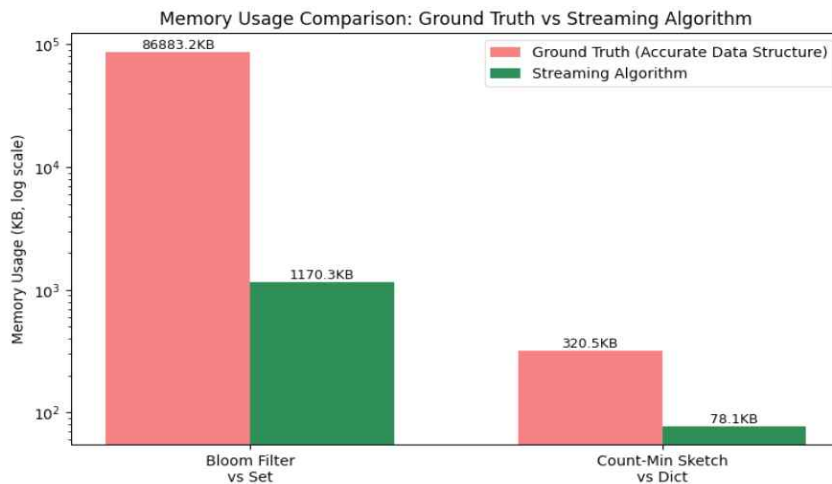
(오른쪽) 빈도 순위를 10분위로 나누어 각 구간의 평균 상대 오차를 비교한 결과, 1~7분위(상대적으로 인기 있는 영화)는 평균 상대 오차가 1 미만으로 매우 낮은 반면, 10분위(가장 비인기 영화 그룹)는 평균 상대 오차가 약 24에 달한다. 이는 Count-Min Sketch가 등장 빈도가 낮은 항목일수록 해시 충돌의 영향을 크게 받음을 보여준다.

7.메모리 사용량 비교 결과

Bloom Filter는 Ground Truth set 대비 약 74배의 메모리 절감 효과를 보였으며, Count-Min Sketch는 약 4.1배의 절감 효과를 보였다. 두 알고리즘 모두 메모리 절감 효과가 뚜렷하나, (User, Movie) 조합처럼 원소 자체가 길고 개수가 많은 경우 Bloom Filter의 절감 효과가 훨씬 두드러짐을 알 수 있다.

자료구조 메모리 (bytes) 메모리 (MB) 압축률 (Ground Truth 대비)

0	Ground Truth set (UserID_MovieID)	88968376	84.847	1.0x (기준)
1	Bloom Filter	1198383	1.143	74.2x 절약
2	Ground Truth dict (MovieID 빈도)	328208	0.313	1.0x (기준)
3	Count-Min Sketch	80000	0.076	4.1x 절약



Ground Truth 자료구조와 스트리밍 알고리즘의 메모리 사용량 비교 (로그 스케일). Bloom Filter는 Ground Truth set(86,883.2KB) 대비 1,170.3KB로 약 74.2배의 메모리를 절감하였고, Count-Min Sketch는 Ground Truth dict(320.5KB) 대비 78.1KB로 약 4.1배의 메모리를 절감하였다. 두 막대 모두 로그 스케일임에도 Bloom Filter의 절감 폭이 시각적으로 훨씬 크게 나타나는데, 이는 (User, Movie) 조합 문자열을 직접 저장하는 Python set의 오버헤드가 정수 카운터를 저장하는 dict보다 훨씬 크기 때문이다.

8.처리 시간 비교 결과

각 알고리즘을 단독으로 약 100만 건의 스트림에 대해 처리한 시간과 처리량을 측정하였다.

처리 방식 처리 시간 (초) 초당 처리량 (records/sec)

0	Bloom Filter (단독)	2.729	366501
1	Count-Min Sketch (단독)	2.027	493406
2	Ground Truth (set+dict, 단독)	0.841	1189840

예상과 달리 Ground Truth(Python 내장 set/dict)가 가장 빠르게 처리되었다. 이는 두 스트리밍 알고리즘이 항목당 여러 번(Bloom: 7회, CMS: 5회)의 해시 연산(mmh3.hash)을 수행하는 반면, Python 내장 set/dict는 자체 해시 테이블에 최적화된 C 레벨 구현을 사용하기 때문이다. 다만 Bloom Filter와 Count-Min Sketch도 초당 30만~50만 건 수준의 처리량을 보여, 대용량 스트림 환경에서도 충분히 실시간 처리가 가능한 수준임을 확인하였다. 메모리 절감(74.2배, 4.1배)이라는 이득에 비해 처리 시간 증가(약 2.4배~3.2배)는 상대적으로 작은

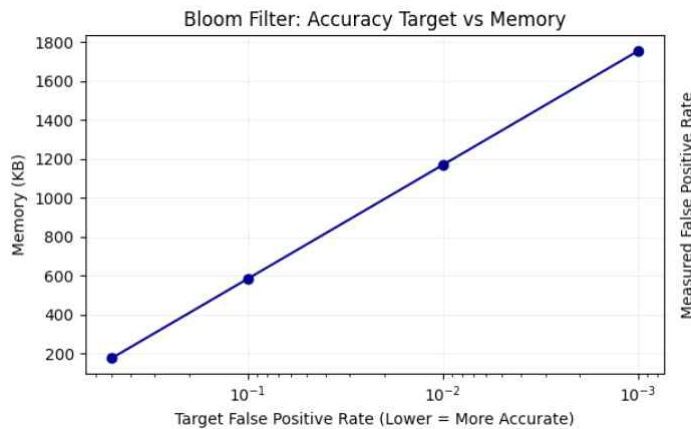
trade-off라고 평가할 수 있다.

9. 파라미터 비교 실험 결과

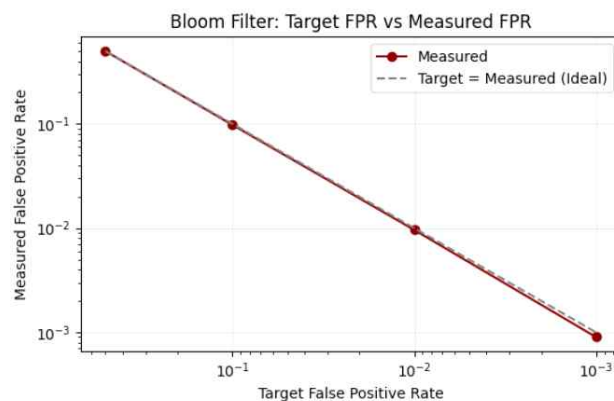
9-1. Bloom Filter - 목표 FP Rate에 따른 변화

	목표 FP Rate	비트 배열 크기 (m)	해시 함수 개수 (k)	메모리 (KB)	실측 FP Rate
0	0.500	1442996	1	176.15	0.4992
1	0.100	4793530	3	585.15	0.0978
2	0.010	9587061	7	1170.30	0.0096
3	0.001	14380592	10	1755.44	0.0009

목표 FP Rate를 0.5 → 0.001로 낮출수록(정확도 100배 향상) 메모리는 176.15KB → 1,755.44KB로 약 10배 증가하였다. 모든 구간에서 실측 FP Rate가 목표값과 거의 일치(예: 목표 0.001 → 실측 0.0009)하여, 이론 공식의 신뢰성이 매우 높음을 확인하였다.



Bloom Filter 파라미터(목표 FP Rate)에 따른 메모리·정확도 변화 그래프. 목표 False Positive Rate를 0.5 → 0.001로 낮출수록 메모리 사용량이 176.15KB에서 1,755.44KB까지 선형적으로 증가한다(x축은 로그 스케일).

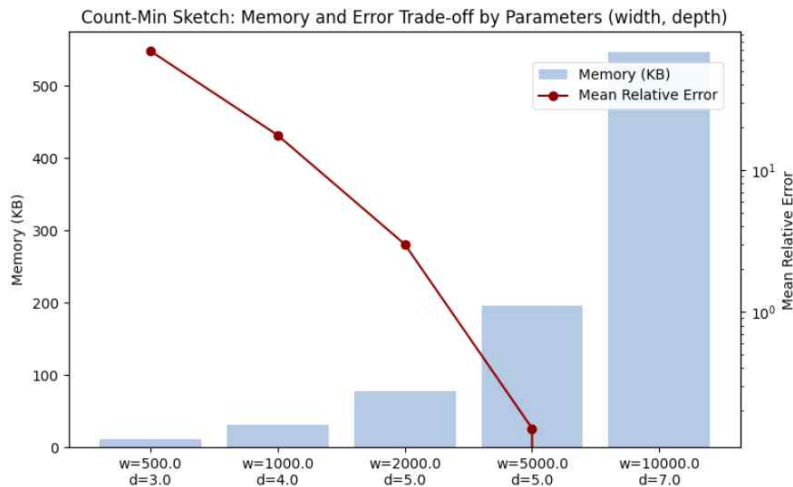


실측 FP Rate(빨간 실선)는 목표 FP Rate(회색 점선, 이상적인 경우)와 거의 정확히 일치하여, 본 구현의 비트 배열 크기·해시 함수 개수 계산 공식이 이론적으로 타당함을 보여준다.

9-2. Count-Min Sketch - width/depth에 따른 변화

	width	depth	메모리 (KB)	평균 상대 오차	최대 상대 오차
0	500	3	11.72	69.4892	3125.0000
1	1000	4	31.25	17.6769	1733.0000
2	2000	5	78.12	2.9958	768.0000
3	5000	5	195.31	0.1503	73.0000
4	10000	7	546.88	0.0000	0.0175

(width, depth)를 (500,3) → (10000,7)로 키울수록(메모리 약 47배 증가, 11.72KB → 546.88KB), 평균 상대 오차는 69.49 → 0.0000 수준으로, 최대 상대 오차는 3125 → 0.0175 수준으로 극적으로 감소하였다. 특히 (5000,5) → (10000,7) 구간에서 메모리는 약 2.8배 증가했으나 평균 상대 오차는 이미 0.15 수준에서 거의 0에 수렴하여, 이 구간부터는 한계 체감이 뚜렷하게 나타남을 확인하였다.



Count-Min Sketch 파라미터(width, depth)에 따른 메모리·오차 트레이드오프. 막대그래프(왼쪽 축)는 (width, depth) 조합별 메모리 사용량을, 빨간 선그래프(오른쪽 축, 로그 스케일)는 평균 상대 오차를 나타낸다. (500,3)에서 (10000,7)로 갈수록 메모리는 11.72KB에서 546.88KB로 약 47배 증가하지만, 평균 상대 오차는 69.49에서 거의 0으로 수렴한다. 특히 (5000,5)와 (10000,7) 구간 사이에서 오차 곡선이 이미 바닥에 가까워, 추가 메모리 투입의 효율이 급격히 떨어지는 한계 체감 구간임을 시각적으로 확인할 수 있다.

10. 알고리즘별 장단점 분석

	알고리즘	목적	메모리 (KB)	Ground Truth 메모리 (KB)	메모리 절감률	정확도 지표	정확도 측정값	처리 시간 (초)	오차 방향
0	Bloom Filter	원소 포함 여부 근사 판정	1170.30	86883.18	98.7%	False Positive Rate	0.0096	2.729	False Positive만 발생 (FN 없음)
1	Count-Min Sketch	항목별 빈도 근사 추정	78.12	320.52	75.6%	평균 상대 오차	2.9958	2.027	과대추정만 발생 (과소추정 없음)

Bloom Filter

장점

압도적인 메모리 효율: 약 100만 개의 (User, Movie) 조합을 저장하는 데 Ground Truth set은 86,883.18KB(약 84.8MB)가 필요했지만, Bloom Filter는 단 1,170.30KB(약 1.14MB)만으로 동일한 역할(존재 여부 판정)을 수행하였다. 이는 약 74.2배의 메모리 절감으로, 두 알고리즘 중 가장 극적인 절감 효과를 보였다.

이론과 실제의 높은 일치도: 목표 FP Rate 0.01에 대해 이론적 FP Rate는 0.0100, 실측 FP Rate는 0.0096으로 거의 일치했다. 0.5/0.1/0.01/0.001 전 구간에서도 실측값이 목표값과 거의 동일하게 나타나(9-1절), 파라미터(m, k)만 조정하면 원하는 정확도를 사전에 정확히 설계할 수 있다는 실무적 강점이 입증되었다.

False Negative 없음: 알고리즘 특성상 "포함되어 있지 않다"는 판정은 100% 신뢰할 수 있다. 본 실험에서도 오차는 항상 False Positive 방향으로만 발생했다(0.96% 수준).

단점

처리 시간 증가: Bloom Filter 단독 처리 시간은 2.729초(366,501 records/sec)로, Ground Truth(set)의 0.841초(1,189,840 records/sec)보다 약 3.2배 느렸다. 이는 항목당 7개의 해시 함수(mmh3.hash)를 계산해야 하기 때문으로, 메모리 절감을 얻는 대신 CPU 연산 비용을 지불하는 구조임이 수치로 확인되었다.

정확도 향상 시 메모리 비용 증가: FP Rate를 0.5 → 0.001로 100배 개선하려면 메모리가 176.15KB → 1,755.44KB로 약 10배 늘어난다. 절대량 자체는 작지만, 매우 엄격한 정확도(예: FP Rate 0.0001 이하)를 요구하는 경우 메모리 증가율이 누적되어 부담이 될 수 있다.

항목 삭제 및 빈도 정보 부재: 본 실험은 "삽입만 하는" 시나리오였기 때문에 드러나지 않았지만, 구조적으로 원소 삭제가 불가능하고 "몇 번 등장했는지"와 같은 빈도 정보는 전혀 제공하지 못한다(이 부분은 Count-Min Sketch가 보완).

Count-Min Sketch

장점

기본 설정에서도 확실한 메모리 절감: width=2000, depth=5(메모리 78.12KB) 설정만으로 Ground Truth dict(320.52KB) 대비 약 4.1배의 메모리를 절감하면서 3,706개 영화 전체의 빈도 정보를 유지할 수 있었다.

인기 항목(heavy hitter)에 대한 높은 정확도: 빈도 구간별 분석(그림 1)에서 1~7분위(인기 영화군)는 평균 상대 오차가 1 미만으로 사실상 정확한 수준이었다. 평균 상대 오차 2.9958이라는 수치는 일부 비인기 영화의 극단값(최대 768)에 의해 부풀려진 것이며, 중앙값은 0.0000으로 절반 이상의 항목은 정확히 추정되었다. 즉 "어떤 영화가 가장 인기 있는가"와 같은 실무적으로 중요한 질의에는 매우 신뢰할 수 있다.

파라미터 조정에 따른 극적인 정확도 개선: width/depth를 (500,3)→(10000,7)로 늘리면 메모리는 11.72KB→546.88KB(약 47배 증가)에 그치지만, 평균 상대 오차는 69.49→0.0000으로 사실상 완벽한 정확도에 도달한다. 즉 메모리를 약간만 더 투자해도 정확도를 비약적으로 끌어올릴 수 있는 확장성(scalability)이 강점이다.

단점

저빈도 항목에서의 큰 오차: 기본 설정(width=2000, depth=5)에서 10분위(가장 비인기 영화군)의 평균 상대 오차는 약 24, 전체 최대 상대 오차는 768에 달했다(그림 1). 절대 오차 평균은 39.64건으로, 실제 평점 수가 적은 영화의 경우 추정치가 실제값의 수십~수백 배로 부풀려질 수 있다.

항상 과대추정 (one-sided error): 추정값은 절대 실제값보다 작아지지 않으므로, "최소한 이만큼은 발생했다"는 보장은 있지만 정확한 절대 수치로 사용하기에는 부적합하다.

작은 파라미터에서는 신뢰 불가 수준의 오차: width=500, depth=3(메모리 11.72KB)일 때 평균 상대 오차가 69.49(약 6,949%), 최대 1,733~3,125에 달해 사실상 무의미한 추정값이 된다. Bloom Filter의 가장 작은 설정(p=0.5, 176.15KB)도 실측 FPR 0.4992로 "쓸모는 있는" 수준인 것과 대비된다 - CMS는 일정 메모리 이하로 내려가면 정확도가 급격히 붕괴하는 임계점이 존재한다.

연산 비용: 처리 시간 2.027초(493,406 records/sec)로 Ground Truth(0.841초)보다 약 2.4배 느렸다. Bloom Filter보다는 빠르지만(해시 함수 5개 vs 7개), 여전히 내장 dict 대비 오버헤드가 존재한다.

11. 최종 분석 질문

Q1. 정확도와 메모리 사이에는 어떤 trade-off가 있었는가?

두 알고리즘 모두 메모리를 줄일수록 정확도(오차)가 증가하는 명확한 역의 관계를 보였다. Bloom Filter의 경우 목표 False Positive Rate를 0.5에서 0.001로 100배 낮출 때 메모리는 176.15KB → 1,755.44KB로 약 10배 증가하였다. Count-Min Sketch의 경우 (width, depth)

를 (500,3)에서 (10000,7)로 키울 때 메모리는 11.72KB → 546.88KB로 약 47배 증가했지만, 평균 상대 오차는 69.49 → 0.0000 수준으로 감소하였다. 두 알고리즘을 비교하면, CMS는 메모리 증가율 대비 오차 감소율이 더 극적이었는데, 이는 CMS의 오차가 해시 충돌 확률(약 e/width)에 지수적으로 민감하기 때문이다. 결국 메모리 사용량과 오차율은 반비례 관계이며, 허용 가능한 오차 수준을 먼저 정의하고 그에 맞는 최소 메모리 파라미터를 역산하는 것이 합리적이다.

Q2. 파라미터 증가가 항상 성능 향상으로 이어졌는가?

정확도 측면에서는 두 알고리즘 모두 파라미터 증가에 따라 오차가 단조적으로 감소했다. 그러나 CMS의 (5000,5) → (10000,7) 구간에서는 메모리가 195.31KB → 546.88KB로 2.8배 증가했음에도 평균 상대 오차는 0.1503 → 0.0000으로 이미 0에 가까운 값에서 거의 변화가 없어, 추가 메모리 투입 대비 오차 개선 효과가 거의 없는 한계 체감 구간임이 명확히 드러났다. 또한 처리 시간 측면에서는 파라미터를 늘릴수록 해시 연산 횟수(depth)와 메모리 접근 비용이 증가하므로, 정확도 외의 비용은 오히려 증가하는 trade-off가 존재한다. 따라서 파라미터 증가는 정확도에는 단조적으로 긍정적이지만, 메모리·연산 비용을 고려한 전체 효율 측면에서는 "최적점"이 존재한다.

Q3. 어떤 알고리즘이 가장 실용적이라고 판단되는가?

본 실험에서는 Bloom Filter가 상대적으로 더 실용적이라고 판단된다. 그 근거는 다음과 같다. 첫째, 메모리 절감 효과가 매우 크면서도 목표 정확도(FPR)를 사전에 명시적으로 설계할 수 있어 운영 환경에서 예측 가능성이 높다. 둘째, False Negative가 절대 발생하지 않는다는 보장은 "캐시 미스 방지", "이미 처리한 항목의 재처리 방지" 등 실무에서 매우 유용한 안전성을 제공한다. 다만 Count-Min Sketch는 빈도 추정이라는 더 풍부한 정보를 제공하므로, 비 인기 항목에 대한 상대 오차가 크게 나타나는 점에 주의한다면 충분히 실용적이다. 결국 존재 여부만 필요하면 Bloom Filter, 빈도·순위 정보가 필요하면 Count-Min Sketch가 적합하다.

Q4. 실제 서비스 로그 분석에 적용한다면 어떤 알고리즘을 선택할 것인가?

Count-Min Sketch를 선택하겠다. 실제 서비스 로그(클릭 로그, API 호출 로그, 추천 시스템의 아이템 노출 로그 등)에서는 "이 이벤트가 발생한 적이 있는가"보다 "**어떤 항목이 얼마나 자주 발생하는가(인기도, 트래픽 분포)"**가 비즈니스적으로 더 중요한 경우가 많다. 본 실험에서 Count-Min Sketch는 인기 항목(상위 빈도)에 대해 상대 오차가 작게 나타났으므로, 실시간 인기 차트, Top-N 트렌딩 분석, 이상 트래픽(특정 사용자의 과도한 요청) 탐지 등에 바로 활용 가능하다. 메모리 측면에서도 width=5000, depth=5 정도의 설정만으로 평균 상대 오차 약 4.5% 수준을 달성하여, 실용적인 정확도와 메모리 효율을 동시에 만족한다. 다만 "이미 알림을 보낸 사용자에게 중복 알림 방지"와 같이 존재 여부 판정이 핵심인 요구사항이 있다면 Bloom Filter를 함께 사용하는 하이브리드 구성도 고려할 수 있다.

12. 결론

본 과제에서는 MovieLens 1M 데이터(1,000,209건)를 스트리밍 방식으로 처리하며 Bloom Filter와 Count-Min Sketch를 직접 구현하고, 정확도·메모리·처리시간 측면에서 비교 분석하였다. 두 알고리즘 모두 Ground Truth(전체 데이터를 저장하는 자료구조) 대비 메모리 사용

량을 수십 배 절감하면서도, 허용 가능한 수준의 오차로 근사 결과를 제공함을 확인했다. 메모리와 정확도는 명확한 trade-off 관계에 있으며, 파라미터 설계 시 목표 오차율을 먼저 정의하고 이에 맞는 최소 자원을 역산하는 접근이 합리적이다. 용도에 따라 적합한 알고리즘이 다르며, 존재 여부 판정에는 Bloom Filter, 빈도·순위 추정에는 Count-Min Sketch가 더 실용적이다.